

Video Transcript - Part 2

Slide 1

Welcome everyone to Part 2 of our AWS Certified Machine Learning Engineer Associate preparation course. My name is Prajan Shrestha, and today we're taking a deep dive into Data Fundamentals and Architecture. Before you can build, train, or deploy machine learning models on AWS, you have to understand the backbone of every ML pipeline: the data itself. How it's structured, how it moves, where it lives, and how it's formatted directly impacts model performance, pipeline efficiency, and AWS cloud costs. Let's jump right into our first module.

Slide 2

In this first section, we'll establish a foundational understanding of data structures, from structured tabular data to unstructured media files and look at the 3 V's of big data that directly drive architectural decisions in AWS Machine Learning pipelines.

Slide 3

Data coming into your AWS ML environment broadly falls into three distinct categories: First, Structured Data. This is data that is highly organized into tabular formats using rigid, predefined schemas with explicit rows and columns. Examples include SQL relational databases, traditional CSV files, and financial transaction logs. Second, Semi-Structured Data. This data contains organizational tags or key-value markers, but lacks a strict tabular schema. Good examples are JSON payloads, XML documents, and NoSQL document stores like Amazon DynamoDB. Finally, Unstructured Data. This data lacks any predefined structural model, yet it makes up over 80 percent of modern enterprise data. Examples include raw images, audio clips, videos, and unstructured PDF documents. Knowing which type you are handling dictates your choice of storage and preprocessing tools in AWS.

Slide 4

To design scalable ML architectures on AWS, you must evaluate data across three core dimensions—the 3 V's: Volume: Refers to the scale and total quantity of generated data, ranging from Gigabytes to Exabytes. Large volumes necessitate scalable, elastic object storage solutions like Amazon S3. Velocity: This is the speed at which data is produced and ingested. It determines whether you process data in periodic batches or stream it in real time using services like Amazon Kinesis or Amazon MSK. Variety: This covers the diverse structural types—structured, semi-structured, and unstructured—that must be unified into a single ingestion pipeline.

Slide 5

Next, let's compare the primary architectural paradigms you'll encounter on AWS: Data Warehouses store primarily structured data using a Schema-on-Write paradigm, where data is transformed before loading. Amazon Redshift is the flagship service here. Data Lakes handle all

data types—structured, semi-structured, and unstructured. They follow a Schema-on-Read model, preserving raw data until query time. In AWS, this is built using Amazon S3 paired with AWS Lake Formation. Data Lakehouses represent a modern hybrid. They combine the low-cost flexibility of a data lake with the unified governance, metadata management, and ACID transaction support of a warehouse—typically implemented using Amazon Redshift Spectrum, S3, and Apache Iceberg.

Slide 6

Looking closer at modern enterprise patterns, let's contrast the Lakehouse architecture with the Data Mesh concept: As shown in the architecture diagram, a Data Lakehouse directly links raw ingestion—whether batch or real-time streaming through an S3 open format storage layer and a metadata management tier. This allows declarative APIs and SQL tools to directly query data for business intelligence, data science, and machine learning without moving it around. On the organizational side, a Data Mesh decentralizes data ownership. Instead of one central team managing a massive data pool, individual domain teams own their data as a product while adhering to federated global governance standards.

Slide 7

When connecting your ML engines and analytics tools to underlying databases, you'll rely on JDBC or ODBC drivers: JDBC (Java Database Connectivity) is platform-independent but language-dependent, built specifically for Java and JVM environments. In AWS ML, JDBC is widely used by Java and Scala frameworks, such as Apache Spark on EMR or AWS Glue ETL jobs. ODBC (Open Database Connectivity) is language-independent but platform-dependent, using native OS C/C++ client drivers. This is standard when connecting BI tools like Amazon QuickSight or Tableau directly to Redshift and Athena.

Slide 8

When designing ingestion pipelines, data typically arrives from three primary operational sources: Raw Logs: Application server output, CloudWatch metrics, and audit logs. REST APIs: Used to pull structured or JSON-formatted payloads from microservices and external web applications. Streaming Data: Continuous, real-time event streams from IoT devices or web clickstreams ingested via Amazon Kinesis or Apache Kafka.

Slide 9

Let's evaluate text-based file formats: CSV is human-readable and universally supported by libraries like Pandas and Amazon SageMaker. However, it lacks embedded schemas and can be slow to parse at scale. JSON excels as a standard interchange format for nested, hierarchical data. Its trade-off is verbosity—repeated key-value labels increase overall file size and reading overhead.

Slide 10

For write-heavy streaming architectures, Apache Avro is the preferred row-based binary format. Because it stores records sequentially in a compact binary structure, it's heavily used with Apache Kafka, Amazon MSK, and Spark. A key feature of Avro is that it embeds its JSON-formatted schema right inside the file header, allowing seamless schema evolution as fields change over time.

Slide 11

For high-performance machine learning reads and analytics, Apache Parquet is the absolute gold standard. Parquet stores data by column rather than row. This allows for aggressive compression and enables Predicate Pushdown—meaning query engines like Amazon Athena or Redshift Spectrum read only the specific columns needed for a model. This cuts query processing times by over 90% and significantly lowers AWS data scanning costs.

Slide 12

To wrap up: mastering data types, storage paradigms, connectivity protocols, and file formats like Parquet and Avro isn't just theory, it's essential for passing the exam and building efficient, cost-effective ML pipelines on AWS. Thank you for your time, and I'll see you in the next one!